

SatQuery AI

A build plan in plain English

Everything you need to decide before the 36-hour finale: which data you can actually get, which models to run, the one training job that matters, how the agent should be wired, and what is most likely to be on fire at hour 30.

THE WHOLE THING IN SIX SENTENCES

You are building a website where someone uploads satellite images, types a question, and gets an answer plus a receipt showing how the answer was produced.

ISRO demands five things. The cheapest way to tick two of them at once is a single small training job on BigEarthNet that eats optical and radar images together.

Do every bit of training **before** the finale. You have never fine-tuned a vision model; doing it for the first time inside a 36-hour clock is how teams end up with nothing.

One dataset is a trap: CDVQA ships question-and-answer text with **no images at all**. The images live somewhere else, behind one Google Drive link. Copy it this week.

Do not try to run two 7-billion-parameter models. Run one small talker plus a few tiny specialists.

Because only the visible receipt gets graded, plain code (not the AI) must choose which model runs.

A note on how this is written: every technical term is explained the first time it shows up. If a section already feels obvious, skip it - the parts about training and about reading satellite files are the ones worth slowing down for, because that is where the unfamiliar work sits.

SECTION 01

What you are actually building

Picture a receptionist at a specialist clinic. Someone walks in holding a couple of scans and a question. The receptionist looks at what they brought, checks it is the right kind of scan and that it is readable, works out what is really being asked, sends it to the right specialist, collects the answer, and hands back a written record of who was consulted and why.

SatQuery AI is that receptionist. The specialists are AI models. The scans are satellite images. The written record is the part ISRO grades.

The words you will keep meeting

TERM	WHAT IT MEANS
VLM (vision-language model)	An AI that can look at a picture and talk about it. ChatGPT with eyes.
Remote sensing	Pictures of Earth taken from satellites. The view is straight down, which is why ordinary AI models are bad at it - they were trained on holiday photos.
Optical image	A normal camera photo from space. Useless at night or under cloud.
SAR image (radar)	Bounces radio waves off the ground and measures what comes back. Works at night and straight through cloud. Looks like grainy grey static to a human eye, which is exactly why it needs careful handling.
Co-registered	Two images lined up so that the same pixel means the same spot on the ground. ISRO says their hidden test images are already lined up - your job is to check that claim, not to do the lining up.
GeoTIFF	An image file that also stores where on Earth it sits, what the pixel size is, and which map projection it uses. This is the format ISRO will hand you.
Bi-temporal	Two pictures of the same place taken at different times, so you can ask what changed.
Fine-tuning	Taking a model somebody else already trained and training it a bit more on your own data so it learns your subject.
LoRA	The cheap way to fine-tune. Freeze the huge model and bolt on a tiny trainable attachment, roughly one percent of the size. Trains on a free GPU, and it still counts as fine-tuning.
VRAM	Memory on the graphics card. Models must fit inside it. This is the constraint that quietly decides your whole architecture.

The five things ISRO demands

Everything in this document exists to tick one of these five boxes. Note that boxes 1 and 4 are ticked by the same piece of work - that is the central trick of this plan.

#	THE DEMAND	WHAT IT ACTUALLY MEANS	WHERE YOU TICK IT
1	Adapt to satellite imagery	Some visual part of your system must be trained on real remote-sensing data. A general AI model straight off the shelf is explicitly rejected.	The 12-channel encoder in Section 04

#	THE DEMAND	WHAT IT ACTUALLY MEANS	WHERE YOU TICK IT
2	Single image	Answer questions about one image. Plus either describe the scene, or point at things the user names.	The small VLM, doing question-answering and captioning
3	Two images, what changed	Given a before and after picture of the same place, describe or answer questions about the difference.	Change detector -> change statistics -> VLM turns those into sentences
4	Optical plus radar together	Pull out information that only appears when you look at both a camera image and a radar image of the same spot.	The same 12-channel encoder, plus its three-way comparison table
5	An agent that orchestrates	Something that reads the question, works out the task, checks the images are valid, picks models, runs them, merges the results, scores its own confidence, and shows its working.	The LangGraph controller in Section 05 - your strongest ground

WHY THIS IS GOOD NEWS FOR YOU

Requirement 5 - the agent - is the heaviest single item on the list, and it is the thing you are already strongest at. The vision work is real but bounded. Play to that: make the orchestration and the receipt genuinely excellent, and keep the model side deliberately small and honest.

SECTION 02

The data: what to download, and the one trap

Good news first: all four datasets ISRO names are genuinely public. No forms, no academic email, no waiting. I checked each source directly.

Bad news: one of them does not contain what you would assume it contains.

THE TRAP

CDVQA is the dataset for the change-analysis requirement, and it has no images in it. The repository holds twelve JSON files - questions and answers only, about 55 MB total. The actual before-and-after pictures come from a completely different dataset called SECOND, distributed through a single Google Drive link with no stated licence.

Google Drive links to research datasets go quota-limited or vanish without warning. If that happens during your finale, a mandatory requirement has no data behind it and there is no substitute.

Download SECOND this week and keep a copy somewhere that is not Google Drive. Then check the filenames actually match what the CDVQA JSON expects, because nobody guarantees they do.

DATASET	WHAT IT IS FOR	WHERE	SIZE	LICENCE	STATUS
BigEarthNet v2.0	The training data. 549,488 matched pairs of camera and radar images over Europe, each labelled with what is on the ground.	Zenodo 10891137	About 110 GB total (59 optical + 51 radar)	CDLA-Permissive	OPEN
VRSBench	Test set for describing images and pointing at objects. 29,614 images.	Hugging Face xiang709/VRSBench	12.5 GB	CC-BY-NC - non-commercial only	OPEN
RSVQA-LR	Small, fast test set for question-answering. 772 images, 77k questions.	Zenodo 6344334	150 MB	CC-BY-4.0	OPEN
RSVQA-HR	Bigger, sharper version of the same. 10,659 images, 1M questions.	Zenodo 6344367	14.4 GB	CC-BY-4.0	OPEN
CDVQA	Test set for change questions. Text only - no images.	GitHub YZHJessica/CDVQA	55 MB	Apache-2.0	NO IMAGES
SECOND	The images CDVQA needs. 4,662 before/after pairs.	captain-whu SCD page, via Google Drive	-	None stated	ONE LINK
Bhoonidhi	ISRO's own portal. Real Cartosat-2S and EOS-04 scenes - the closest thing to the hidden test set.	Bhoonidhi portal	per scene	ISRO open data	LOGIN

Three jobs to do this week

- 1 **Mirror SECOND.** Download it, put a second copy on an institutional drive or a physical disk, and verify the filenames line up with the CDVQA JSON files. Aim for at least 95 percent matching before you trust it.
- 2 **Cut BigEarthNet down.** 110 GB will not fit in a free cloud notebook and does not need to. Take 20,000 to 40,000 matched optical/radar pairs, spread evenly across the 19 land-cover categories, using the official geographic split. Drop the snow, cloud and shadow patches - the dataset ships a metadata file that flags them. Upload the result as a Kaggle dataset so training sessions mount it instantly instead of re-downloading.
- 3 **Get real ISRO scenes.** Register on Bhoonidhi and pull at least one Cartosat-2S optical scene and one EOS-04 radar scene. Every benchmark dataset is a tidy little 512-pixel tile; a real ISRO scene is a large multi-band file with its own band order, number format and quirks. If your code has only ever seen tidy tiles, it will break on the one input that counts.

MENTION THIS IN YOUR SUBMISSION

VRSBench is non-commercial, and the small Qwen model recommended later carries a research-only licence. That is completely fine for a hackathon and for research evaluation. Just say so plainly in your write-up rather than implying you have built something commercially deployable. Judges notice when a team is straight about this.

SECTION 03

Which models to run

The instinct is to grab the most impressive model for each job. Resist it. Your real constraint is graphics-card memory, and it bites much sooner than you expect.

A 7-billion-parameter model needs roughly 14 GB of VRAM at full precision. Two of them will not fit on a 16 GB card. You can squeeze them by loading one, unloading it, and loading the next - but that adds 15 to 30 seconds to every single query, which will wreck a live demo in front of judges.

THE RULE

One talker, several tiny specialists.

Keep exactly one vision-language model loaded at all times - a small one. Around it, put two or three models under a gigabyte each that do one narrow job extremely well and hand back plain numbers. The talker turns those numbers into sentences.

This is cheaper, faster, more reliable, and - usefully - much easier to explain in a receipt, because each specialist's contribution is a separate visible line.

MODEL	SIZE	WHAT IT DOES	VRAM FULL / SQUEEZED	VERDICT
remote-sensing-Qwen2.5-VL-3B	3B	Questions, descriptions, pointing at objects	~7 GB / ~3.5 GB	Runs on ordinary libraries with no special forks. This is your main model. PRIMARY
GeoChat-7B	7B	Same three jobs, better quality	~14 GB / ~6 GB	Strong and well-known, but twice the memory and an older codebase. BACKUP
TEOChat-7B	7B	Change questions across time	~14 GB / ~6 GB	Exactly the right capability on paper, but built on an older video codebase that fights modern libraries. Only if you are ahead of schedule. STRETCH
EarthDial-4B	4B	Multi-band imagery dialogue	~9 GB / ~4 GB	MIT licensed and handles multispectral data. Worth an hour of evaluation. EVALUATE
Your 12-channel encoder	~25M	Optical + radar fusion, land cover	under 1 GB	You build this. See Section 04. Ticks two ISRO requirements at once. BUILD
Siamese change detector	~15M	Before/after difference mask	under 1 GB	Small, standard, trains in an hour on SECOND. BUILD
GeoPixel-7B	7B	Pixel-perfect outlines	over 20 GB	Beautiful and completely unnecessary. Boxes already satisfy the requirement. CUT

Squeezed means 4-bit quantisation - storing the model's numbers at lower precision. It roughly halves memory use for a small quality cost, and it is one line of configuration.

Where your instinct will be wrong

For the change requirement you will want TEOChat, because it is the scientifically correct answer and it is designed for exactly this. Do not start there. Build the change path out of parts instead:

- 1 The small change detector compares the two images and produces a mask - a picture showing which pixels changed.
- 2 Plain Python turns that mask into numbers: percentage of area changed, which land-cover types turned into which others, where the biggest changed regions sit.
- 3 The VLM reads those numbers and writes the answer or the description.

This is more robust under time pressure, it produces a visual output you can draw on screen for free, and every step in the middle is a separate line in your receipt - which is what gets graded. Keep TEOChat configured as an optional extra you can switch on if you finish early.

SECTION 04

The training you must do (and the one that matters)

This is the section to read slowly, because it is the part furthest from what you already know.

ISRO's wording is: *at least one visual or VLM component must be fine-tuned on BigEarthNet or other open remote-sensing training data*. Read that carefully - a fine-tuned **visual** component satisfies it. It does not have to be the language model. That matters enormously, because training a small image classifier is ordinary, well-documented work, whereas fine-tuning a vision-language model for the first time is not.

Job A: the 12-channel encoder

This is the single most valuable thing you will build. Two to four hours on a free GPU, and it ticks two of ISRO's five boxes.

What you are training

A standard image model (ResNet-50 or a small vision transformer) that looks at a satellite patch and predicts which of 19 land-cover categories are present - forest, water, urban fabric, arable land and so on. Several can be true at once, which is called multi-label classification. It is one of the most standard tasks in computer vision and there are a thousand tutorials.

The trick that makes it count twice

An ordinary photo has 3 channels: red, green, blue. Instead of that, stack **10 optical bands from Sentinel-2 plus 2 radar bands from Sentinel-1** into a single 12-channel input, and feed all twelve to one model at once.

That one change is what makes this genuine *fusion*. The model learns features that depend on camera and radar evidence together, rather than producing two separate opinions that you glue together afterwards with string. A domain expert on the judging panel will ask you exactly this question, and this is the answer.

WHY BIGEARTHNET, SPECIFICALLY

BigEarthNet is the only large public archive of camera and radar images that are already lined up pixel-for-pixel over the same places. That is almost certainly why ISRO names it in the same breath as a Cartosat optical plus RISAT radar test set - it is the only realistic way to practise for the hidden evaluation. One training run therefore gives you: the mandatory fine-tune (requirement 1), a real optical-plus-radar component (requirement 4), and a small fast model that can fact-check the VLM's claims about land cover. Nothing else in this project has that return on effort.

The table that proves it works

Train three versions and report all three: optical only, radar only, and both together. The fused version must beat both single-sensor versions. That single comparison table **is** your evidence for requirement 4 - not a paragraph claiming fusion happened, an actual measured number showing it helped. It comes free with the run you were doing anyway.

Hardware and time

- **Use Kaggle, not Colab.** Kaggle gives 30 GPU-hours a week, 12-hour sessions, and datasets that mount without re-downloading. Free Colab sessions get reclaimed mid-run, which will happen to you at the worst moment.

- **Budget:** roughly 2 to 4 hours on a free T4 or P100 for 30,000 patches over about 10 passes through the data.
- **Starting point:** load ImageNet weights and widen the first layer from 3 channels to 12. You are not training from scratch - that would take days.

Job B: the LoRA fine-tune (attempt only after Job A is done)

Once the encoder is finished and its numbers are written down, adapt the language model so it speaks in the vocabulary of your pipeline and answers in the format the benchmarks expect.

Feed it roughly 12,000 to 15,000 training examples, mixed:

- About 6,000 questions generated automatically from BigEarthNet's labels - "is there water in this patch?", "what is the dominant cover?", and questions that explicitly reference the radar signal.
- About 4,000 from the RSVQA training split, so it learns the answer formatting the benchmark expects.
- About 3,000 image descriptions and object references from VRSBench.
- About 1,000 examples that turn change statistics into fluent sentences.

Settings: LoRA rank 16, freeze the vision half, load the base model in 4-bit, one pass through the data. Reported memory use for this exact model is about 8.9 GB, which fits a free 16 GB T4 with room to spare. Budget 6 to 10 hours.

Use LLaMA-Factory or Unsloth. Do not write your own training loop. For a first vision-model fine-tune, a config file is the difference between shipping and debugging tensor shapes at three in the morning.

MEASURE BEFORE YOU TRAIN

Fix a test set of 500 questions and score the **un-trained** model on it first. Then you can report a real before-and-after difference.

If the fine-tune does not improve anything, say so honestly and ship Job A. A documented negative result reads far better to a technical panel than a vague claim that adaptation happened.

THE HARD RULE

Both training jobs finish before the finale. They ship as frozen files committed to your repository. Training inside the 36 hours, with no prior experience of it, is the single most reliable way to reach hour 30 with nothing running.

SECTION 05

The agent: how the controller should work

This is your home ground, so the guidance here is mostly about one design decision that people get wrong.

THE DECISION THAT MATTERS

ISRO says only the **visible execution trace** is graded. The AI's internal reasoning text is explicitly not evaluated.

So: **let the AI work out what is being asked, but let plain code decide which model runs.**

If the AI freely picks its own tools, your receipt records a choice nobody can reproduce - run it twice and you may get different paths. If ordinary code filters your model registry on typed rules, the receipt records a rule ID, every candidate considered, and why each one was rejected. The second is an audit trail. The first is just a transcript.

The nine steps

STEP	WHAT IT DOES
ingest	Open every uploaded file and record what it is: checksum, map projection, pixel size, image dimensions, number format, how many bands, where on Earth it sits, and which sensor it probably came from.
validate	Run the checks in Section 06. Record the ones that pass too - a receipt that only lists failures proves nothing.
(branch)	If any check is fatal, stop here and return a clear named reason. Never crash.
classify	The AI reads the question and returns a structured label: is this a question, a description request, a pointing request, a change query, or a cross-sensor query? Plus how confident it is.
(branch)	If confidence is low, or the number of images contradicts the task, ask the user a clarifying question instead of guessing.
plan	Plain code filters the model registry: which models can do this capability, on these sensors, with this many images? Rank the survivors. Record every candidate and why each was rejected.
(branch)	If nothing matches, fall back to a declared alternative - for example, radar missing, so run the optical-only path - and flag that clearly.
preprocess	Prepare the images for the chosen model: pick bands, convert radar brightness properly, tile oversized scenes, resize. Log every transformation. This is where quality silently disappears, so make it visible.
execute	Run the models. Independent ones run in parallel. Record which model, which version, what settings, which device, how many milliseconds.
fuse	Merge text and shapes into one answer: the prose, the boxes or masks in both pixel and map coordinates, and which model contributed which claim.
confidence	Score the answer from named parts, never one mystery number.
respond	Assemble the answer and the full receipt, and stream it out.

Scoring confidence honestly

Report four parts, each between 0 and 1, and show which one is weakest:

- **Input quality** - how well the images line up, how much cloud or empty data, whether the pixel sizes match.
- **Task certainty** - how clearly the classifier picked one task over the others.
- **Model fitness** - the chosen model's own score on the closest matching benchmark.
- **Agreement** - does the small specialist back up what the VLM said? If the VLM claims water is present and your land-cover encoder disagrees, confidence should visibly drop and the receipt should name the disagreement.

That last one is what makes the whole confidence score credible rather than decorative, and it costs nothing extra - you already ran both models.

THE DEMO MOMENT

Stream each step to the screen as it completes, over server-sent events. The judge watches validation pass, sees the task get classified, sees candidate models get filtered out one by one, sees the models run - all before the answer appears.

A receipt that only shows up after the answer reads as an excuse written afterwards. A receipt that builds live in front of someone reads as an agent genuinely working. Same data, completely different impression. Aim for the first trace line on screen within half a second of pressing enter.

SECTION 06

Reading satellite files without breaking things

Old advice says you need Anaconda to install geospatial Python libraries on Windows. That is no longer true. **Plain** `pip install rasterio` **now works** - it ships Windows packages with the underlying geospatial engine included, for Python 3.10 and up. Do not install standalone GDAL.

Your entire toolkit: **rasterio** to read files and their geography, **pyproj** to compare map projections, **shapely** to check whether two image footprints overlap, and **numpy** for the maths. That is it.

What to check on every upload

Each check returns a name, a status, what was expected and what was found. Only fatal ones stop the run.

For each image on its own

- Does it open at all, and what format is it? **FATAL**
- Is it a GeoTIFF? Plain PNG or JPEG is allowed only for the benchmark datasets. **FATAL**
- Does it know where it is on Earth? A missing map projection means it is not georeferenced. **WARN**
- How many bands, and what number format? This is how you work out which sensor it is.
- What fraction of pixels are empty or no-data? Over 40 percent should reduce confidence.
- How big is it? Oversized scenes must go through the tiling path rather than being squashed.

For a pair of images

- Same map projection, or at least convertible?
- How much do the footprints overlap? Divide the shared area by the smaller image's area. You need 0.80 or better to honestly claim they are lined up; below 0.30, stop. **FATAL**
- Do the pixel sizes match? If one is more than twice the other, one must be resampled - and you must record that you did it.
- **Are they on the same grid?** Compare the corner offsets against the pixel size. Overlapping footprints only prove they cover the same area; matching grids prove they are truly aligned. This is the check that separates a real co-registration test from a lazy one.
- For cross-sensor tasks, are the two images actually from different sensors?
- Which came first? Read the acquisition date from the file tags where available.

Work out the sensor from the pixels, not the filename

- **Radar:** 1 or 2 bands, decimal numbers, enormous range between darkest and brightest, histogram heavily skewed to one side.
- **Ordinary colour photo:** 3 bands, whole numbers 0 to 255.
- **Multispectral optical:** 4 to 13 bands, larger whole numbers.

Use the filename only as a supporting hint, and log both what you inferred and what the filename suggested. Filenames lie; pixels do not.

THE MISTAKE THAT QUIETLY RUINS THE RADAR REQUIREMENT

If you stretch a radar image the obvious way - find the darkest pixel, find the brightest, scale everything in between - **you get an almost entirely black image**. A handful of very bright reflectors (metal roofs, corner structures) dominate the whole range and crush everything else to nothing.

Feed that to a vision model and it will confidently describe an empty scene. Your cross-sensor requirement is then technically running and producing nonsense, and nothing in your logs will say so.

The correct sequence: convert to decibels using 10 times log-base-10, guarding against zeros; clip to the 2nd and 98th percentile rather than the true minimum and maximum; scale that to 0-255; copy across three channels. For dual-polarisation radar, use the two polarisations and their ratio as three false-colour channels - far more informative, and it costs one extra line.

Prove it once, visually. Render both versions of a real EOS-04 scene side by side in a notebook and keep that image. It is a genuinely persuasive slide, and it shows you understand the sensor rather than just calling a library.

SECTION 07

What will be on fire at hour 30

Ordered by how much of the deliverable each one takes down, multiplied by how likely it is. Every single one has something you can do about it right now.

#	WHAT BREAKS	WHY IT HAPPENS	DO THIS NOW
1	Your Python environment collapses on Windows CRITICAL	These model repositories pin old library versions and assume a component called flash-attention that does not build cleanly on Windows. Two models with conflicting requirements in one environment is a multi-hour hole with no partial credit at the end of it.	Build and freeze the whole setup inside WSL2 or Docker today. Run one real query through each candidate model. Save the exact version list. Choosing the small Qwen model largely dissolves this problem, because it runs on ordinary current libraries with no special fork.
2	The hidden ISRO images break your file reader CRITICAL	Every benchmark is a neat 512-pixel tile. The real test is full Cartosat and RISAT scenes with unfamiliar band ordering, number formats, no-data conventions and sheer size. Code that has only ever seen neat tiles fails on the one input that is graded.	Register on Bhoonidhi, pull real scenes, and make them permanent test files you run against every time. Build the tiling path early - you will be handed a whole scene, not a patch.
3	Your sensor fusion turns out to be cosmetic CRITICAL	The requirement most likely to look satisfied while not being satisfied. Running an optical model and a radar model separately and pasting their sentences together is not extracting complementary information, and a domain judge will ask precisely that.	Ship the 12-channel encoder with its three-way comparison. One table where fused beats optical-only and radar-only is the whole defence, and it is free with the training run you were already doing.
4	The SECOND download is unavailable HIGH	The change-analysis images sit behind one Google Drive link with no licence and no mirror. If it is quota-limited during the finale, a mandatory requirement has no data.	Download and mirror it today. Verify the filenames match the CDVQA files before assuming they do.
5	The fine-tune trains fine but improves nothing HIGH	A clean run that fails to beat the base model leaves requirement 1 defensible only on paper.	Job A is the insurance and is near-certain to work - it is ordinary supervised learning on a labelled dataset. Fix your test set and score the base model before training, so you are measuring a real difference rather than hoping for one.
6	You run out of graphics memory mid-demo HIGH	Two 7-billion models plus specialists will not fit on 16 GB. Swapping models in and out adds 15 to 30 seconds per query.	Commit to the one-talker budget now and measure peak memory on your heaviest query type. This is an architecture decision, not a tuning problem - deciding it late means rewriting the registry.
7	The frontend eats your last eight hours HIGH	The classic failure for a backend-strong team: the agent works beautifully and the demo cannot show it.	Build the React shell before the finale against a fake data stream - upload box, question box, live trace panel, image overlay. The trace panel is the only custom component that carries marks; everything else can be an unstyled component library.

#	WHAT BREAKS	WHY IT HAPPENS	DO THIS NOW
8	Model downloads over venue wifi HIGH	Tens of gigabytes of model files over contested hackathon wifi, at the worst possible moment.	Pre-download everything to local disk and carry it physically. Set the offline environment flag and confirm the system starts with networking switched off.
9	Change answers come out poor HIGH	CDVQA answers span six land-cover types with counting and comparison questions. A weak change mask poisons every answer downstream.	Score the change path on CDVQA's test split early. If the mask is the bottleneck, the fix is a better detector - a small, bounded, well-understood problem - not a better prompt.
10	The live demo dies on stage HIGH	Any of the above, in front of judges, with a clock running.	Build a replay mode that plays back saved runs at realistic speed, and record a walkthrough. Demo from the replay; offer live as the encore.

SECTION 08

The schedule

The finale is an **integration window, not a build window**. Anything with an open-ended failure mode - environments, training, downloads - happens before it.

Before the finale

WHEN	GOAL	WHAT GETS DONE
Week 1	Secure the inputs	Mirror SECOND and cross-check filenames. Pull RSVQA and VRSBench. Register on Bhoonidhi and get real Cartosat and EOS-04 scenes. Build and publish the BigEarthNet subset. Freeze the environment in WSL2 or Docker and get one real query running end to end.
Week 2	Train and freeze	The 12-channel encoder plus its three-way comparison table. The change detector on SECOND. Score the base VLM on your fixed test set to establish the before number. Then, only if all of that is done and written up, the LoRA fine-tune.
Week 3	Skeleton and shell	File reading and the full validation suite, tested against the real ISRO scenes. The model registry, its loader and its rule-based selector, with unit tests. The React shell and live trace panel against a fake stream. Pre-download every model file and verify the system boots with networking off.

The 36 hours

WHEN	GOAL	WHAT GETS DONE
H0 - H6	Wire the graph	All nine steps with real signatures and empty bodies. The trace accumulator working end to end. One hardcoded path that returns a complete receipt.
H6 - H14	Single-image path	Questions, descriptions and object pointing running through the real model, with boxes drawn on screen. Checkpoint: requirement 2 is demonstrable.
H14 - H22	Change path	Two-image validation, change detector, change statistics, VLM turning them into sentences. Checkpoint: requirement 3 is demonstrable.
H22 - H28	Cross-sensor path	Optical/radar pair validation, correct radar brightness handling, the fused encoder in the loop, an answer that names what each sensor contributed. Checkpoint: requirement 4 is demonstrable.
H28 - H32	Freeze	Confidence breakdown, receipt export, error paths. Feature freeze at H32, no exceptions. Anything unfinished gets cut and named honestly in the submission.
H32 - H36	Demo	Replay mode, recorded walkthrough, results tables assembled, README and slides. Rehearse twice.

WHY THIS ORDER

Requirements 2, 3 and 4 are sequenced from most certain to least certain, and each one ends at a point where you could stop and still demo something complete.

If you run late, you lose the cross-sensor path last rather than losing everything partially. And even then, the encoder from Job A still evidences requirement 4 in your written report, because the comparison table exists whether or not its screen path got finished.

SECTION 09

Things you still need to find out

- **What bands does Cartosat-2S actually supply in the hidden set?** Panchromatic only, or the four-band multispectral product? This changes what your optical branch receives and whether the encoder needs an extra adapter layer. Answer it by looking at real files from Bhoonidhi.
- **What processing level is the RISAT / EOS-04 data?** Whether the radar arrives already calibrated or as raw counts determines whether your decibel conversion is correct or applied twice. Read the product metadata rather than assuming.
- **Does EarthDial have a released radar checkpoint?** The paper mentions radar in its training mix, but only the colour, multispectral and methane checkpoints appear to be published. If a radar one exists it is a strong shortcut for requirement 4 - worth an hour to confirm.
- **What format should the receipt be in?** ISRO requires an auditable trace but does not specify a schema. Ship JSON plus a readable rendered view, and state your schema explicitly in the README.
- **What happens when the question is about a whole scene rather than a tile?** Aggregating across tiles is an unspecified design decision. The simplest defensible answer: tile it, run each tile, combine with counts and locations, and record the tiling in the receipt.

IF YOU ONLY DO FOUR THINGS THIS WEEK

1. Mirror the SECOND dataset somewhere safe.
 2. Freeze a working environment in WSL2 or Docker with one model answering one real question.
 3. Pull real Cartosat-2S and EOS-04 scenes from Bhoonidhi and open them in rasterio.
 4. Train the 12-channel encoder and write down the three-way comparison.
- Those four remove roughly eighty percent of the risk in this project, and none of them requires the finale to have started.

Dataset sizes, licences, sources and model specifications in this document were checked against primary sources on 30 August 2026. Memory figures and GPU-hour estimates are planning numbers, not measurements - treat them as the right order of magnitude, not as promises.